

建立及連結 AdWords 和 Merchant Center 子帳戶

來源：<https://codelabs.developers.google.com/codelabs/shopping-account-linking?hl=zh-tw>

步驟數：8

在本程式碼研究室中，您將建立解決方案，使用 AdWords API 和 Content API For Shopping 新建由 AdWords 管理員帳戶和 Merchant Center 多重客戶帳戶管理的新帳戶。接著連結新的子帳戶，讓新 Merchant Center 子帳戶管理的產品，能再用於由新 AdWords 子帳戶建立的購物廣告活動。

目錄

1. [1. 簡介](#)
2. [2. 開始設定](#)
3. [3. 設定驗證方法](#)
4. [4. 連線至 API](#)
5. [5. 建立新的受管理 AdWords 帳戶](#)
6. [6. 建立新的 Merchant Center 子帳戶](#)
7. [7. 接受來自 AdWords 帳戶的連結](#)
8. [8. 主題變化](#)

1. 簡介

在本程式碼研究室中，您將瞭解如何使用 [Content API for Shopping](#) 和 [AdWords API](#) 的基本概念，並建構同時使用這兩者的應用程式。具體來說，您將建構指令列應用程式，用於建立及連結 AdWords 帳戶和 Merchant Center 帳戶。

課程內容

- 如何建立由管理員帳戶管理的 AdWords 帳戶。
- 如何建立由多重客戶帳戶管理的 Merchant Center 帳戶。
- 如何從 Merchant Center 帳戶要求連結至 AdWords 帳戶。
- 如何在 AdWords 帳戶中接受待處理的 Merchant Center 連結。

軟硬體需求

- [AdWords 管理員帳戶](#)
 - [Merchant Center 多重客戶帳戶](#)
 - Java 7 以上版本
 - [Maven](#)
 - [程式碼範例](#)
 - 文字編輯器 (建議使用 Eclipse 或 IntelliJ 等可解讀 Maven 專案的 IDE)
-

2. 開始設定

下載程式碼

點選下方連結即可下載這個程式碼研究室的所有程式碼：

將下載的 ZIP 檔案解壓縮。這會解壓縮根資料夾 (`shopping-account-linking-master`)，其中包含 Maven 專案和您需要的所有資源。請特別注意下列子目錄：

- `src/main/java` 是 Maven 專案的來源根目錄，內含程式碼架構，供您在其中作業。
- `src/main/java/solution` 包含完成的解決方案。

安裝必要套件並建構

如果您使用 **Eclipse** 或 **IntelliJ** 等支援 **Maven** 的 IDE，可以將解壓縮的資料夾匯入為 Maven 專案，然後正常編譯專案。

如果您是從指令列使用 **Maven**，可以執行下列指令來擷取必要套件，並從解壓縮專案的根資料夾 (`shopping-account-linking-master`) 編譯專案：



```
mvn compile
```

步驟 3 · 約 10 分鐘

3. 設定驗證方法

在這個步驟中，我們不會撰寫任何程式碼，但會設定包含 AdWords API 和 Content API for Shopping 適當驗證權杖的檔案。

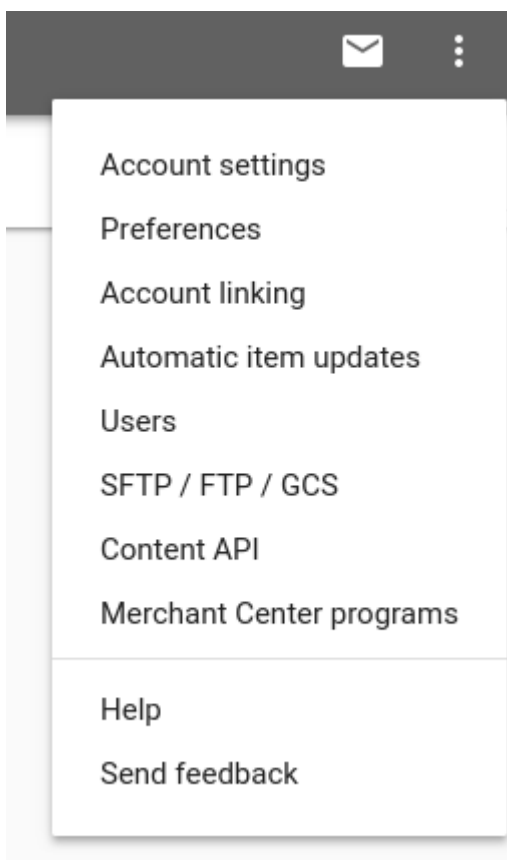
設定 AdWords API 驗證

本程式碼研究室使用的憑證載入方式與用戶端程式庫相同，因此如果您已透過管理員帳戶使用 [Google Ads API Java 專用用戶端程式庫](#)，應該就已完成設定。否則，請按照[入門](#)步驟操作，開始使用適用於 Java 的 Google Ads API 用戶端程式庫。

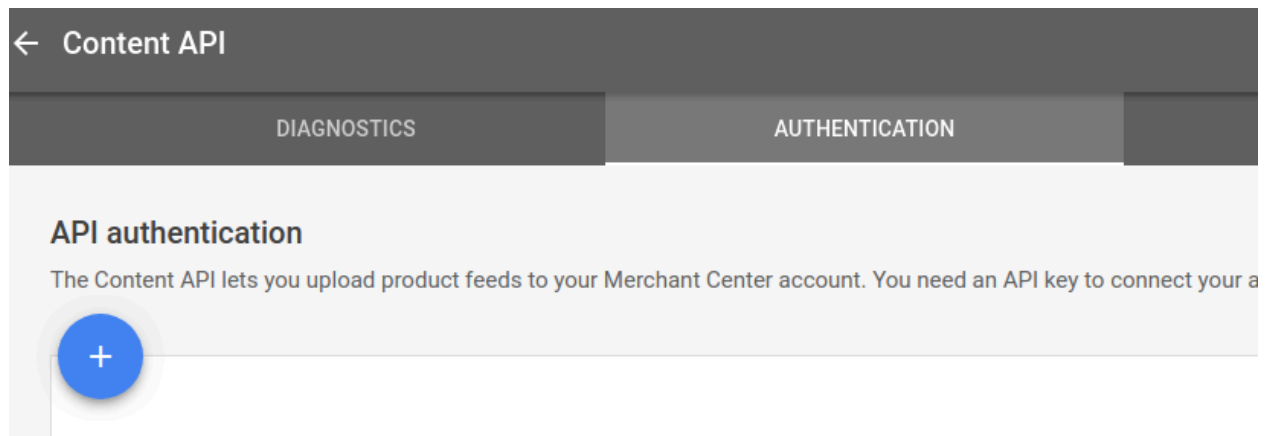
設定 Content API 驗證

如果您還沒有服務帳戶金鑰，請按照下列步驟操作：

1. 前往多重客戶帳戶的 [Merchant Center](#)，然後從溢位選單中選取「Content API」：



2. 選取「驗證」，然後點選藍色的「+」按鈕：



3. 接受 Google Cloud Platform 和 Google API 服務條款後，瀏覽器會自動下載含有新服務帳戶金鑰的 JSON 檔案。

現在請按照操作說明，[設定購物範例的服務帳戶驗證](#)。也就是說，服務帳戶金鑰副本應位於主目錄的下列路徑：`shopping-samples/content/service-account.json`。除非您想在完成本程式碼研究室後試用範例，否則不需要設定範例設定。

立即測試

現在驗證權杖已位於正確位置，請嘗試執行範例。如果您在指令列使用 Maven，請執行下列指令：

```
mvn compile
```

注意：如果您變更 `src/main/resources` 中的任何檔案，即使沒有進行任何程式碼變更 (例如這個步驟)，也請先重新編譯，再執行範例，將變更的資源複製到目標。

```
mvn exec:java -Dexec.mainClass="SolutionRunner"
```

如果收到有關未提供工作階段物件的錯誤訊息，表示驗證權杖已就位且運作正常！否則，您收到的錯誤訊息應會說明哪些憑證無法運作，以及要修正哪個檔案。

重要事項：我們提供的框架假設你的 Content API 服務帳戶只列為多重客戶帳戶的使用者。如果收到錯誤訊息，指出服務帳戶可存取多個帳戶，請按照上述步驟，取得專為多重客戶帳戶使用的服務帳戶金鑰。

步驟 4 · 約 5 分鐘

4. 連線至 API

您現在已取得要使用的兩個 API 的有效驗證權杖，接下來請開始填寫實際程式碼。首先，我們會使用驗證權杖建立工作階段物件。在後續步驟中，我們會使用這些工作階段物件，存取各個 API 提供的各種服務和方法。

建立 Content API 工作階段物件

如要建立 Content API 工作階段，請建構 `ShoppingContent.Builder` 物件，然後使用該物件建構適當的 `ShoppingContent` 物件。幸好，建構前者所需的一切都已在程式碼架構中提供，因此我們只需要像這樣將其整合在一起：

SolutionRunner.java

```
// TODO(sessions): Create a ShoppingContent object using ShoppingContent.Builder.
contentApiSession =
    new ShoppingContent.Builder(httpTransport, jsonFactory, contentApiCredential)
        .setApplicationName("Linking AdWords and Merchant Center Accounts Codelab")
        .build();
```

注意：這個和後續程式碼範例中的初始註解，已存在於程式碼架構中。請找出這些檔案，瞭解應加入哪些程式碼行。

此外，我們也會省略這些程式碼片段中使用的 AdWords API 和 Content API 類別所需的所有匯入陳述式。如果您使用的 IDE 無法自動新增正確的匯入項目，請檢查 `solution` 套件中的程式碼，找出所需的匯入項目。

嚴格來說，設定應用程式名稱並非必要，但這會顯示如何在呼叫 `build()` 方法前，透過 `ShoppingContent.Builder` 物件設定任何所需選項。

建立 AdWords API 工作階段物件

同樣地，也有用於建構 `AdWordsSession` 物件的 `AdWordsSession.Builder` 類別。主要差異在於，我們不會直接在建構工具上設定設定選項，而是使用 `fromFile()` 方法，從上一步設定的 `ads.properties` 檔案載入選項。

SolutionRunner.java

```
// TODO(sessions): Create a AdWordsSession object using AdWordsSession.Builder.
adWordsSession =
    new AdWordsSession.Builder()
        .fromFile()
        .withOAuth2Credential(adwordsOAuth2Credential)
        .build();
```

立即測試

如果您是從指令列執行 Maven 專案，請使用與上一節相同的指令重建及執行專案：

```
mvn compile
```

```
mvn exec:java -Dexec.mainClass="SolutionRunner"
```

這次您應該完全不會收到任何錯誤，但也不會收到任何有趣的輸出內容。我們會在呼叫 API 來建立及連結新帳戶時，加入該值。

5. 建立新的受管理 AdWords 帳戶

現在我們已建立 API 工作階段物件，接下來將逐步建立要連結的帳戶。我們將從 AdWords 開始，在管理員帳戶下建立測試帳戶。

注意：本節中的所有程式碼片段都會依序新增至架構程式碼的相同位置。因此，我們只會顯示第一個檔案中的註解 (// TODO...)，標示註解在第一個檔案中的位置。因此，在後續步驟中，如果看到沒有簡介註解的程式碼片段，請直接接在先前的片段後面。

存取 ManagedCustomerService

在 AdWords API 中，我們首先會使用靜態 `getInstance()` 方法擷取 `AdWordsServices` 類別的例項，藉此存取各種可用服務。接著，我們可以使用這個執行個體，透過 `get()` 方法為這些服務建立用戶端。這個方法會採用兩個引數：要建立用戶端的工作階段，以及所需服務的介面。

SolutionRunner.java

```
// TODO(newAWaccount): Using the ManagedCustomerService, create a new testing AdWords account
// under the given manager account.

AdWordsServicesInterface adWordsServices = AdWordsServices.getInstance();

ManagedCustomerServiceInterface managedCustomerService =
    adWordsServices.get(adWordsSession, ManagedCustomerServiceInterface.class);
```

在此，我們存取 `ManagedCustomerService`，以便從指定管理員帳戶管理 AdWords「客戶」(帳戶)。

指定新的帳戶設定

首先，我們會建立 `ManagedCustomer` 物件，其中包含新帳戶的設定。在本程式碼研究室中，我們會建立測試帳戶，並將幣別設為美元，時區則設為美國西岸時區。

SolutionRunner.java

```
Random rand = new Random();
long run = rand.nextLong();

ManagedCustomer newAdWordsAccount = new ManagedCustomer();
newAdWordsAccount.setName(String.format("AdWords Account Created by Run %d", run));
newAdWordsAccount.setTestAccount(true);
newAdWordsAccount.setCurrencyCode("USD");
newAdWordsAccount.setDateTimeZone("America/Los_Angeles");
```

我們也會建立隨機數字，並加入帳戶名稱。這是為了將我們在此建立的 AdWords 帳戶，與稍後建立的 Merchant Center 帳戶相符，以便在解決方案完成後，目視檢查並確認兩者確實已連結。

建立新的客戶帳戶

如要實際建立新帳戶，我們會使用 `ManagedCustomerOperation` 指定 `ADD` 作業：

SolutionRunner.java

```
ManagedCustomerOperation operation = new ManagedCustomerOperation();
operation.setOperand(newAdWordsAccount);
operation.setOperator(Operator.ADD);
```

接著，我們會使用 `ManagedCustomerService` 物件的 `mutate()` 方法執行作業。這個方法會採用要執行的作業陣列，但我們這裡只想要執行單一作業。`mutate()` 方法的結果是包含 `ManagedCustomer` 清單的值；在這裡，這會是包含一個客戶 (也就是我們建立的新帳戶) 的清單。我們會擷取該新帳戶的 ID 以供日後使用，並列印出來，以便在解決方案的輸出內容中查看。

SolutionRunner.java

```
ManagedCustomerReturnValue result =
    managedCustomerService.mutate(new ManagedCustomerOperation[] {operation});
Long adWordsId = result.getValue()[0].getCustomerId();
System.out.printf("Created new AdWords account %d\n", adWordsId);
```

立即測試

如先前所述，請嘗試執行解決方案。如果您是透過指令列使用 Maven：

```
mvn compile
```

```
mvn exec:java -Dexec.mainClass="SolutionRunner"
```

如果一切順利，您應該還是不會看到任何錯誤，這次我們會看到您建立的新 AdWords 帳戶 ID。前往 [AdWords](#) 網站查看管理員帳戶，您應該也會看到新帳戶列在其中！

注意：此時，每次執行解決方案都會產生新帳戶。解決方案放送完畢後，您可以透過 [AdWords](#) 網站取消任何已建立的 AdWords 帳戶。畢竟，我們為這些帳戶採用的命名架構，可讓您輕鬆查看哪些帳戶是由程式碼建立！

步驟 6 · 約 5 分鐘

6. 建立新的 Merchant Center 子帳戶

在這個步驟中，我們會建立 Merchant Center 子帳戶，並連結至上一個步驟中建立的 Google Ads 帳戶。建立子帳戶後，我們不必另外要求連結，因為我們已有對應 AdWords 帳戶的 ID，因此可以在建立子帳戶時要求連結。

指定新子帳戶的設定

與 AdWords API 不同，`Account` 模型類別的設定項會傳回物件，因此我們可以在新的 `Account` 物件上，將呼叫鏈結至這些設定項。我們也會使用在建立 Google Ads 帳戶時產生的隨機號碼，做為新 Merchant Center 帳戶的名稱。

SolutionRunner.java

```
Account newMcAccount = new Account()
    .setName(String.format("Merchant Center Account Created by Run %d", run))
    .setAdwordsLinks(
        ImmutableList.of(
            new AccountAdwordsLink()
                .setAdwordsId(BigInteger.valueOf(adWordsId))
                .setStatus("active")));
```

如本步驟簡介所述，由於我們已有新客戶帳戶的 AdWords ID，現在可以將該 ID 新增至新子帳戶的 `AdwordsLinks` 清單。建立新的子帳戶後，系統會自動要求建立這個連結，並在 AdWords API 中提供。

建立新的子帳戶

在 Content API 中，我們會呼叫工作階段物件的 `accounts()` 方法來存取 `Accounts` 服務，然後直接呼叫 `insert()` 方法，而不是設定作業物件。這個方法會採用兩個引數：要建立新子帳戶的多重客戶帳戶 ID，以及包含所需設定的 `Account` 物件：

SolutionRunner.java

```
newMcAccount = contentApiSession.accounts().insert(mcaId, newMcAccount).execute();

System.out.printf("Created new Merchant Center account %s\n", newMcAccount.getId());
```

`insert()` 方法會傳回 `Account` 物件，其中包含新子帳戶的設定。我們覆寫原始的 `Account` 物件，因為傳回的版本包含重要資訊：新的子帳戶 ID。我們會將該 ID 列印在解決方案的輸出內容中，因此可以執行解決方案，然後在 Merchant Center 中驗證新的子帳戶是否存在。

立即測試

如先前所述，請嘗試執行解決方案。如果您是透過指令列使用 Maven：

```
mvn compile
```

```
mvn exec:java -Dexec.mainClass="SolutionRunner"
```

如果一切順利，您應該還是不會看到任何錯誤，這次我們會看到新 AdWords 帳戶和新 Merchant Center 帳戶的 ID。在多重客戶帳戶的 [Merchant Center](#) 中，查看新的子帳戶。

注意：與 AdWords 相同，每次執行解決方案都會產生新的 Merchant Center 帳戶。解決方案執行完畢後，你可以透過 [Merchant Center](#) 刪除建立的任何子帳戶。

7. 接受來自 AdWords 帳戶的連結

在上一個步驟中，我們建立了新的 Merchant Center 子帳戶，同時要求連結至新的 AdWords 帳戶。在這個步驟中，我們將使用 AdWords API 接受連結要求，完成整個程序。

存取 CustomerService

如同先前，我們會使用 `AdWordsServices` 類別取得 `CustomerService` 的用戶端。不過，在建立用戶端之前，請先變更 AdWords 工作階段物件，以便日後使用新的受管理帳戶，而非管理員帳戶。畢竟，Merchant Center 帳戶要求連結的是受管理帳戶，不是管理員帳戶。

SolutionRunner.java

```
// TODO(acceptLink): Using the mutateServiceLinks method in CustomerService, accept the
// proposed link between the new AdWords account and the new Merchant Center account.

adWordsSession.setClientCustomerId(adWordsId.toString());

CustomerServiceInterface customerService =
    adWordsServices.get(adWordsSession, CustomerServiceInterface.class);
```

指定要求的連結

就像建立新的 AdWords 帳戶時一樣，我們會建立包含連結設定的 `ServiceLink` 物件，然後建立描述所需作業的 `ServiceLinkOperation` 物件。在這裡，我們要將待處理的服務連結帶到 `MERCHANT_CENTER` 帳戶，並將其 `SET` 到 `ACTIVE`。對於 `serviceLinkId` 設定，我們將使用剛建立的 Merchant Center 帳戶 ID，因為這會用於 Google Ads 中的服務連結 ID。

SolutionRunner.java

```
ServiceLink serviceLink = new ServiceLink();
serviceLink.setServiceLinkId(newMcAccount.getId().longValue());
serviceLink.setLinkStatus(ServiceLinkLinkStatus.ACTIVE);
serviceLink.setServiceType(ServiceType.MERCHANT_CENTER);

ServiceLinkOperation op = new ServiceLinkOperation();
op.setOperator(Operator.SET);
op.setOperand(serviceLink);
```

接受連結

最後，我們會呼叫 `CustomerService` 物件的 `mutateServiceLinks()` 方法來執行作業。與先前一樣，這項作業會採用服務連結作業陣列。這次，這個方法會直接傳回 (可能已變更) 服務連結的清單，因此我們只要疊代該清單，即可輸出解決方案的結果。當然，由於我們只指定單一作業，因此您只會預期輸出內容中會顯示單一連結。

SolutionRunner.java

```
ServiceLink[] mutatedServiceLinks =
    customerService.mutateServiceLinks(new ServiceLinkOperation[] {op});
for (ServiceLink mutatedServiceLink : mutatedServiceLinks) {
    System.out.printf(
        "Service link with service link ID %d, type '%s' updated to status: %s.%n",
        mutatedServiceLink.getServiceLinkId(),
        mutatedServiceLink.getServiceType(),
        mutatedServiceLink.getLinkStatus());
}
```

立即測試

如先前所述，請嘗試執行解決方案。如果您是透過指令列使用 Maven：

```
mvn compile
```

```
mvn exec:java -Dexec.mainClass="SolutionRunner"
```

如果一切順利，您應該還是不會看到任何錯誤，這次我們也會看到服務連結已更新為有效的附註。檢查 [AdWords](#) 和 [Merchant Center](#)，確認帳戶確實已連結。

8. 主題變化

恭喜您完成本程式碼研究室！現在您已擁有可正常運作的解決方案，接下來請看幾個範例，瞭解如何修改或擴充解決方案，進一步使用本程式碼研究室介紹的 API。

額外抵免額：更新現有 Merchant Center 帳戶，要求連結 AdWords

在程式碼研究室中，我們巧妙地先建立 AdWords 帳戶，這樣在建立 Merchant Center 帳戶時，就能使用 AdWords 帳戶的資訊要求連結。不過，如果 Merchant Center 帳戶已存在，則需要更新設定。請嘗試變更程式碼，先建立 Merchant Center 帳戶，然後在建立 AdWords 帳戶後返回，更新設定以要求連結。

注意：建議使用 `Accounts.update` 或 `Accounts.patch` 方法。事實上，您可以試著使用每個版本，瞭解兩者之間的差異！

重要事項：呼叫 `Accounts.insert` 後，可能無法立即使用新的 Merchant Center 帳戶。在新帳戶可用之前，嘗試存取或修改帳戶時會收到 401 HTTP 錯誤。購物範例的 `AccountWorkflow.java` 中，有一個使用指數輪詢重試方法的範例，該範例會使用 `BaseWorkflowSample.java` 中的 `retryFailures` 方法。

額外學分：擷取 AdWords 和 Merchant Center 帳戶資訊，驗證連結建立作業

目前應用程式只會將 API 呼叫未發生錯誤視為成功。請嘗試擴充範例，檢查新 Merchant Center 和 AdWords 帳戶的連結資訊，確認連結確實有效。

注意：在 Content API 方面，建議查看 `Accounts.get` 方法。在 AdWords 端，您會想使用 `CustomerServiceInterface` 物件的 `getServiceLinks` 方法。

世界就像寶藏

如果想到其他可進行的變更，不妨試試看！如需構想的參考程式碼，請參閱 [Google 購物範例](#) 和 [Google Ads Java 用戶端程式庫來源](#) 中的 `examples` 目錄。